

Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	SWTID-2026-5245
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation is consistent in api/index.py; HTML is cleanly nested.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear separation into api/, templates/, static/, and model/ folders.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Partial	temperature, humidity, rainfall, crop, confidence are clear; N, P, K, le, pred are short/abbreviated.
4	Function / Method Names	Functions are descriptively named	Yes	home() and predict() clearly describe their routes' purpose.
5	Code Comments	Inline and block comments explain logic	Yes	Inline comments and docstrings have been added to explain input parsing, model loading, and the prediction workflow in api/index.py.
6	Modular Design	Code is split into reusable functions/modules	Partial	Model loading, input parsing, and prediction all sit inside one file/route rather than separate reusable functions.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Line 'app = app' at the end of index.py is redundant and can be removed.
8	Error Handling	Exceptions and errors are handled gracefully	No	No try/except around form value parsing or model prediction; invalid input would raise an unhandled error.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	crop_model.pkl (trained ML model)	Python / scikit-learn	Loaded once in api/index.py and used for every /predict request	High
2	label_encoder.pkl	Python / scikit-learn	Used in predict() to decode the model's numeric output back into a crop name	High
3	index.html form template	HTML5 / CSS3 / Jinja2	Serves as the main input form rendered by the home() route	Medium
4	style.css	CSS3	Shared styling for index.html and result.html	Medium
5	predict() route logic	Python / Flask	Core prediction logic tied to the /predict endpoint	Low – currently coupled to Flask/templates, not extracted into a standalone reusable function

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clean folder organization and consistent indentation; minor spacing-style inconsistencies (e.g., missing spaces around '=' in variable assignments).
Readability	5	Variable and function names are mostly clear, but single-letter names (N, P, K, le) reduce readability for newcomers.
Reusability	4	Model and encoder artifacts are cleanly separated and reusable; the prediction logic itself is not yet extracted into standalone, reusable functions.
Documentation / Comments	5	Inline comments and docstrings now explain the prediction pipeline, input parsing, and model loading, significantly improving maintainability.
Overall Score	5	A functional, well-organized small project with good documentation after adding comments; remaining improvements are input validation/error handling and removing the redundant 'app = app' line.

